



Création et utilisation de la base de données

Julie LORIAU SENSENBRENNER



Laplace Immo

Contexte du projet

PROJET DATAImmo

Le projet « DATAImmo » est mis en place au sein de l'entreprise Laplace Immo, réseau national d'agences immobilières.

Ce projet consiste à créer un modèle se basant sur données des transactions immobilières et foncières en France et qui sera mis à disposition des agences du réseau,

Le but de mieux prévoir le prix de vente des biens immobiliers et accompagner les clients.

Les étapes sont les suivantes :

- A partir des données collectées, modification de la BDD
- Implémentation des tables dans la BDD
- Réalisation des requêtes demandées afin d'analyser le marché

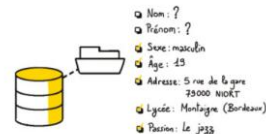


La stratégie de sauvegarde et la conformité RGPD

Les 5 grands principes du RGPD

1. Cartographier et tenir le registre :

- Création d'un dictionnaire des données avec application des contraintes
- Création d'un schéma relationnel précisant les liaisons de façon visuelle



2. Minimisation et conservation :

- Ne conserver que les données utiles au projet et pour une durée limitée
- Un tri des données nécessaires a été effectué selon leur intérêt pour le projet

3. Information et Consentement :

- Travail effectué à partir de données brutes extraites de sites en accès libre
- Consentement de collecte des données reçu lors de la signature des actes notariés

4. Sécurité et Confidentialité :

- Mise à disposition du réseau Laplace Immo, du modèle final d'amélioration de la prédiction des prix de vente
- Conservation de la base au sein du service de la direction générale pour le seul personnel autorisé

5. Désignation d'un DPO (Data Protection Officer)

- Le traitement de données personnelles est inhérent au métier d'agent immobilier, Laplace Immo dispose forcément déjà d'un DPO

Les données initiales

- Données transmises par Clara Daucourt, CTO de Laplace Immo
 - DVF : données issues du site des demandes de valeurs foncières (opendata),
 - Données de l'INSEE comprenant le recensement des populations,
 - Données de data.gouv relatives au référentiel géographique de la France (DROM et Collectivités d'Outre-Mer inclus),
- Analyse des fichiers excel afin de comprendre les données présentes,
- Vérification des colonnes directement dans les fichiers excel :
 - Vides (articles du CGI, lot 2 à 5 et surfaces Carrez afférentes, identifiant local,...),
 - Inutiles selon le besoin métier (CODARR, CODCAN, nature culture, nature culture spéciale, ...)
 - Non adaptées aux requêtes (Surface terrain, code postal, nature mutation,...)
 - Contraire à la RGPD (Nom de l'acquéreur).

L'extrait du dictionnaire des données

- Organisation du dictionnaire des données selon les colonnes conservées dans un premier temps,
- Mise en place d'un onglet par table en lien avec le schéma relationnel dans un second temps,
- Correction du dictionnaire suite à l'import des données dans chaque table

DICTIONNAIRE DES DONNÉES - Table Vente

CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	RÈGLE DE GESTION	RÈGLE DE CALCUL
id_vente	ID dans la base de données	Integer		Elémentaire	Ne doit pas être nul	Auto incrementation
date_mutation	Date de la mutation (vente dans ce fichier) au format AAAA/MM/JJ (norme Iso 8601)	Varchar		Elémentaire		
valeur_foncier	Montant ou évaluation lors de la mutation (prix net vendeur TTC)	Numeric	10.2	Elémentaire		
Id_bien	ID dans la base de données	Varchar	20	Concatenation	Ne doit pas être nul	dep_code + com_code + préfixe + section + n° plan + lot

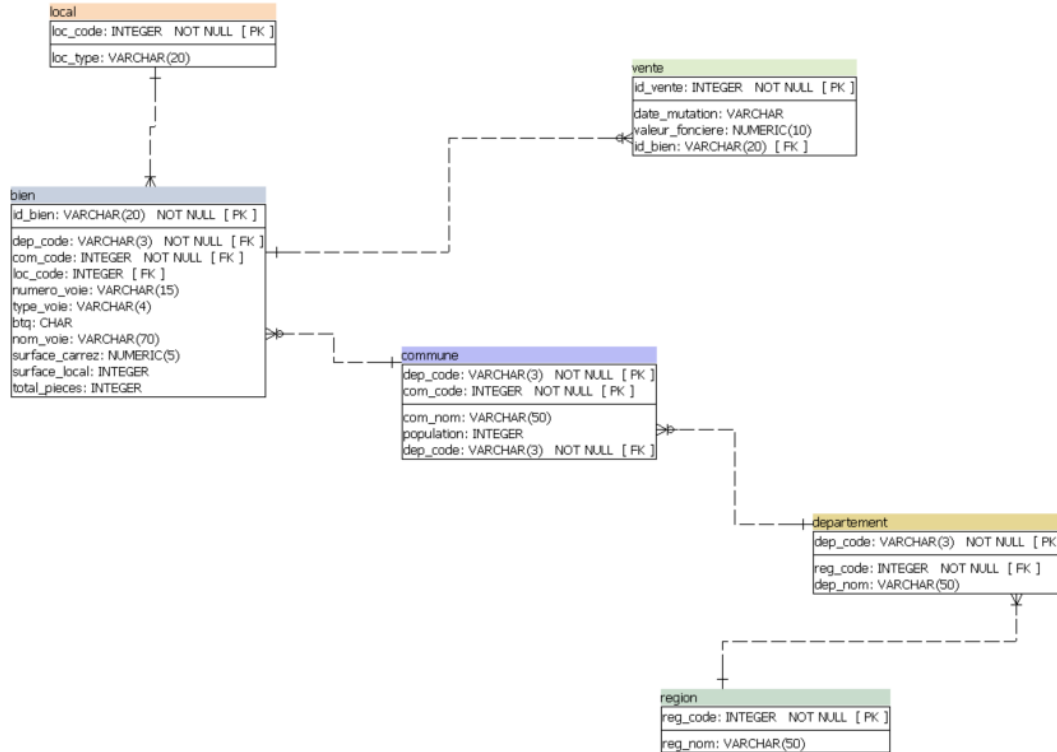
DICTIONNAIRE DES DONNÉES - Table Departement

CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	RÈGLE DE GESTION	RÈGLE DE CALCUL
dep_code	Code représentant le numéro du département PK	Varchar	3	Elémentaire	Ne doit pas être nul	
dep_nom	Nom du département	Varchar	50	Elémentaire		
reg_code	FK de la table region	Integer		Elémentaire	Ne doit pas être nul	

DICTIONNAIRE DES DONNÉES - Table Bien

CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	RÈGLE DE GESTION	RÈGLE DE CALCUL
Id_bien	ID dans la base de données	Varchar	20	Concatenation	Ne doit pas être nul	dep_code + com_code + préfixe + section + n° plan + lot
dep_code,com_code	Code représentant le code de la commune (INSEE)	Varchar	5	Elémentaire	Ne doit pas être nul	
loc_code	Code du local 1 ou 2 FK table local	Integer	1	Elémentaire		
numero_voie	Numéro des rues	Varchar	15	Elémentaire		
type_voie	Plusieurs valeurs (rue, avenue, chemin, etc.)	Varchar	4	Elémentaire		
btq	Indice de répétition	Char	1	Elémentaire		
nom_voie	Nom de la rue	Varchar	70	Elémentaire		
Surface_carrez	Surface du bien selon mesures Loi Carrez	Numeric	5.2	Elémentaire		
Surface_local	Surface réelle correspondant à la surface au sol entre les murs ou séparations et arrondie au mètre carré inférieur. Somme des surfaces réelles du local et de la surface des dépendances,	Integer	6	Elémentaire		
Total_piece	Nombre de pièces principales	Integer	5	Elémentaire		

Le schéma relationnel normalisé : Power Architect



FORMES NORMALES :

1NF :

- Colonnes atomiques (une valeur indivisible par colonne)
- Une clé primaire par table
- Pas de colonnes répétitives

2 NF :

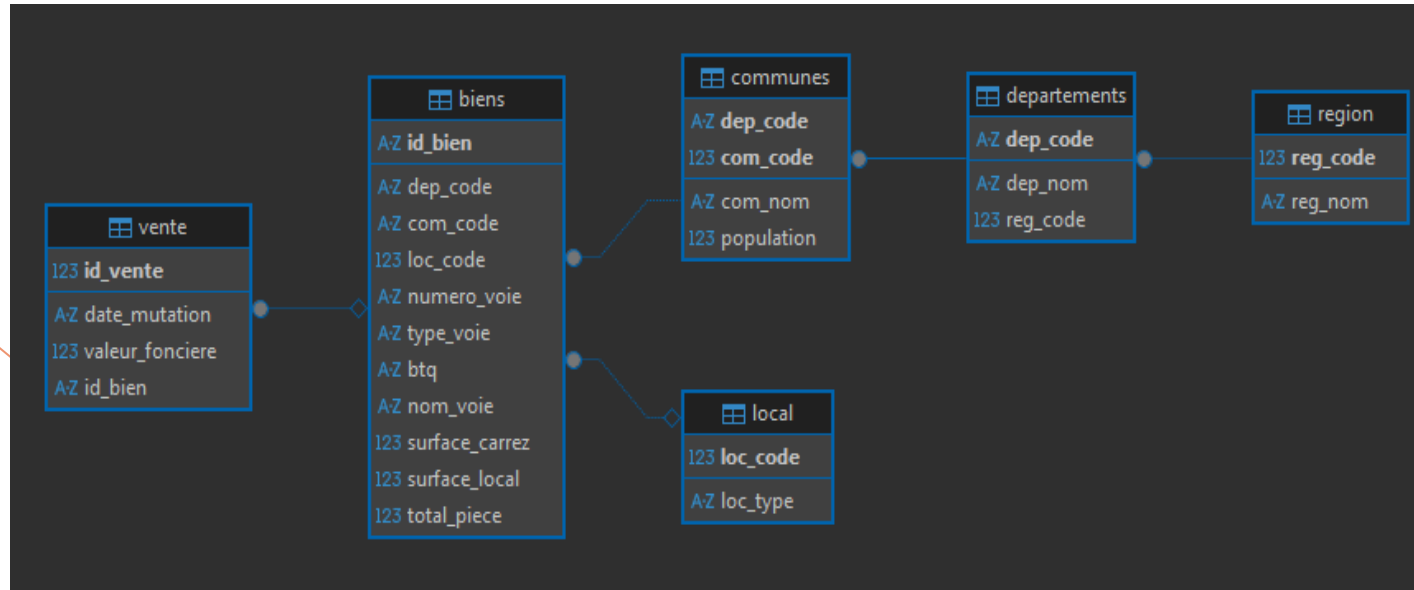
- Tables en 1NF
- Clés simples sauf la PK de commune (permet de créer le code INSEE de la commune)

3 NF :

- Tables en 2NF
- Pas de dépendances transitives

La base de données avec les tables créées

- Choix d'importer les csv bruts, avec modification de noms à l'import
- Création des tables et import des données, en SQL, selon le schéma relationnel



Les requêtes qui permettent de démontrer le bon chargement des données

-- Création table region

```
CREATE TABLE region(  
  reg_code INTEGER PRIMARY KEY NOT NULL,  
  reg_nom TEXT);
```

-- Insertion données table region

```
INSERT INTO region  
SELECT DISTINCT ref_geo.reg_code,  
ref_geo.reg_nom  
FROM ref_geo;
```

-- Création table départements

```
CREATE TABLE departements(  
  dep_code TEXT PRIMARY KEY NOT NULL,  
  dep_nom TEXT,  
  reg_code INTEGER NOT NULL,  
  FOREIGN KEY (reg_code) REFERENCES  
  region(reg_code));
```

-- Insertion données table departements

```
INSERT INTO departements  
SELECT DISTINCT ref_geo.dep_code,  
ref_geo.dep_nom, ref_geo.reg_code  
FROM ref_geo;
```

-- Création table communes

```
CREATE TABLE communes(  
  dep_code TEXT NOT NULL,  
  com_code INTEGER NOT NULL,  
  com_nom TEXT,  
  population INTEGER,  
  PRIMARY KEY (dep_code, com_code),  
  FOREIGN KEY (dep_code) REFERENCES  
  departements(dep_code));
```

-- Insertion données table communes

```
INSERT INTO communes  
SELECT DISTINCT  
  dep_code,  
  CODCOM AS com_code,  
  com_nom,  
  population  
FROM donnees_communes;  
SELECT COUNT (*) FROM communes ;
```

-- Création table local

```
CREATE TABLE local(  
  loc_code INTEGER NOT NULL PRIMARY KEY,  
  loc_type TEXT);
```

-- Insertion données table local

```
INSERT INTO local  
SELECT DISTINCT "Code type local" AS loc_code,  
"Type local" AS loc_type  
FROM Valeurs_foncieres;
```

-- Création table biens

```
CREATE TABLE biens(  
  id_bien TEXT PRIMARY KEY NOT NULL,  
  dep_code TEXT NOT NULL,  
  com_code TEXT NOT NULL,  
  loc_code INTEGER,  
  numero_voie TEXT,  
  type_voie TEXT,  
  btq TEXT,  
  nom_voie TEXT,  
  surface_carrez NUMERIC,  
  surface_local NUMERIC,  
  total_piece INTEGER,  
  FOREIGN KEY (dep_code, com_code) REFERENCES  
  communes(dep_code, com_code),  
  FOREIGN KEY (loc_code) REFERENCES local(loc_code)  
);
```

-- Insertion données table biens

```
INSERT INTO biens(id_bien, dep_code, com_code,  
loc_code, numero_voie, btq, type_voie, nom_voie,  
surface_carrez, surface_local, total_piece)  
SELECT DISTINCT  
  "Code departement" || "Code commune" ||  
  COALESCE("Préfixe de section", '') ||  
  COALESCE("Section", '') || COALESCE("No plan",  
  '') ||  
  COALESCE("1er lot", ''),  
  "Code departement" AS dep_code,  
  "Code commune" AS com_code,  
  "Code type local" AS loc_code,  
  "No voie" AS numero_voie,  
  "B/T/Q" AS btq,  
  "Type de voie" AS type_voie,  
  "Voie" AS nom_voie,  
  "Surface Carrez du 1er lot" AS surface_carrez,  
  "Surface réelle bati" AS surface_local,  
  "Nombre pièces principales" AS total_piece  
FROM Valeurs_foncieres;
```

-- Création table vente

```
CREATE TABLE vente(  
  id_vente INTEGER PRIMARY KEY AUTOINCREMENT,  
  date_mutation TEXT,  
  valeur_fonciere NUMERIC,  
  id_bien TEXT,  
  FOREIGN KEY (id_bien) REFERENCES biens(id_bien)  
);
```

-- Insertion données table vente

```
INSERT INTO vente (date_mutation, valeur_fonciere,  
id_bien)  
SELECT  
  "Date mutation" AS date_mutation,  
  CAST("Valeur fonciere" AS DECIMAL),  
  b.id_bien  
FROM Valeurs_foncieres v  
JOIN biens b ON TRIM(v."Code departement" ||  
  TRIM(v."Code commune") ||  
  COALESCE(v."Préfixe de section", '') ||  
  COALESCE(v."Section", '') ||  
  COALESCE(v."No plan", '') ||  
  COALESCE(v."1er lot", '')) = b.id_bien;  
SELECT COUNT(id_vente) FROM vente;
```

-- Contrôle des résultats

```
SELECT COUNT (*) FROM  
donnees_communes ;
```

Résultat : 34 991

```
SELECT COUNT (*) FROM  
Valeurs_foncieres ;
```

Résultat : 34 169

SELECT COUNT (*) FROM region ;

Résultat : 19

```
SELECT COUNT (*) FROM departements ;
```

Résultat : 109

```
SELECT COUNT (*) FROM communes ;
```

Résultat : 34 991

```
SELECT COUNT (*) FROM local;
```

Résultat : 2

```
SELECT COUNT(*) FROM biens;
```

Résultat : 34 169

```
SELECT COUNT(id_vente) FROM vente;
```

Résultat : 34 169



Requêtes SQL et résultats

Préalable à la mise en œuvre des requêtes

Création d'une View (vue) intitulée « jointure » dans le but d'éviter d'insérer les « join » complexes à chacune des requêtes

```
CREATE VIEW jointure AS
SELECT *
FROM vente as v
JOIN biens b ON v.id_bien=b.id_bien
JOIN communes c ON b.com_code=c.com_code AND
b.dep_code=c.dep_code
JOIN departements d ON c.dep_code=d.dep_code
JOIN region r ON d.reg_code=r.reg_code
JOIN local l ON b.loc_code = l.loc_code
```

Intérêt : Meilleure maintenabilité en cas de changement ou ajout de tables

Contrôle d'unicité

```
--Vérification de l'unicité de la clé  
primaire
```

```
SELECT DISTINCT  
"Code departement" || "Code commune" ||  
COALESCE("1er lot", ' ') ||  
COALESCE("Préfixe de section", ' ') ||  
COALESCE("Section", ' ') ||  
COALESCE("No plan", ' ') AS id_bien  
FROM Valeurs_foncières  
GROUP BY id_bien  
HAVING COUNT(*) > 1;
```

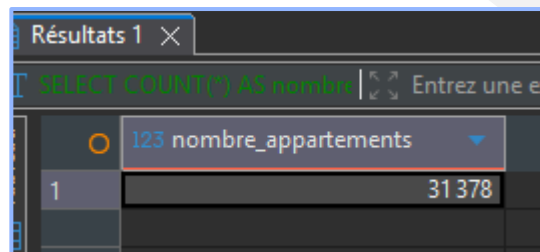
```
--Vérification du nombre de vente par  
bien
```

```
SELECT  
id_bien,  
COUNT(*) as nb_ventes  
FROM jointure  
GROUP BY id_bien  
HAVING nb_ventes > 1  
ORDER BY nb_ventes DESC;
```

REQUETE 1 :

Nombre total d'appartements vendus au 1er semestre 2020

```
SELECT COUNT(*) AS  
nombre_appartements  
FROM jointure j  
WHERE loc_type="Appartement"  
AND date_mutation BETWEEN  
"2020/01/01" AND "2020/06/30";
```



The screenshot shows a database interface with a window titled 'Résultats 1'. The SQL query is visible in the top bar: `SELECT COUNT(*) AS nombre`. Below the query, a table displays the results. The table has two columns: an index column and a column named 'nombre_appartements'. The first row shows the value '1' in the index column and '31 378' in the 'nombre_appartements' column.

	123 nombre_appartements
1	31 378

REQUETE 2 :

Le nombre de ventes d'appartement par région pour le 1er semestre 2020

```
SELECT
reg_nom AS Region,
COUNT(*) AS nombre_appartements
FROM jointure j
WHERE loc_type="Appartement"
AND date_mutation BETWEEN
"2020/01/01" AND "2020/06/30"
GROUP BY reg_nom
ORDER BY nombre_appartements DESC;
```

	AZ Region	123 nombre_appartements
1	Ile-de-France	13 995
2	Provence-Alpes-Côte d'Azur	3 649
3	Auvergne-Rhône-Alpes	3 253
4	Nouvelle-Aquitaine	1 932
5	Occitanie	1 640
6	Pays de la Loire	1 357
7	Hauts-de-France	1 254
8	Grand Est	984
9	Bretagne	983
10	Normandie	862
11	Centre-Val de Loire	696
12	Bourgogne-Franche-Comté	376
13	Corse	223
14	Martinique	94
15	La Réunion	44
16	Guyane	34
17	Guadeloupe	2

REQUETE 3 :

Proportion des ventes d'appartements par le nombre de pièces

```
SELECT DISTINCT
  total_piece,
  COUNT(*) AS nb_ventes,
  ROUND(100.0 * COUNT(*) / (SELECT
    COUNT(*) FROM jointure WHERE
    loc_type="Appartement"), 2) ||
  " %" AS proportion
FROM jointure
WHERE loc_type = "Appartement"
GROUP BY total_piece
ORDER BY total_piece;
```

	123 total_piece ▼	123 nb_ventes ▼	AZ proportion ▼
1	0	30	0.1 %
2	1	6739	21.48 %
3	2	9783	31.18 %
4	3	8966	28.57 %
5	4	4460	14.21 %
6	5	1114	3.55 %
7	6	204	0.65 %
8	7	54	0.17 %
9	8	17	0.05 %
10	9	8	0.03 %
11	10	2	0.01 %
12	11	1	0.0 %

REQUETE 4 :

Liste des 10 départements où le prix du mètre carré est le plus élevé

```
CREATE VIEW prix_m2_dep AS
SELECT
dep_nom,
ROUND(AVG(Valeur_fonciere /surface_carrez ),
2) AS prix_m2
FROM jointure
GROUP BY dep_nom
ORDER BY dep_nom ASC;

SELECT dep_nom, prix_m2
FROM prix_m2_dep
ORDER BY prix_m2 DESC
LIMIT 10;
```

	AZ dep_nom ▼	123 prix_m2 ▼	
1	Paris	12 083,66	
2	Hauts-de-Seine	7 300,21	
3	Val-de-Marne	5 427,42	
4	Haute-Savoie	4 780,96	
5	Alpes-Maritimes	4 755,07	
6	Seine-Saint-Denis	4 385,3	
7	Yvelines	4 275,12	
8	Rhône	4 099,82	
9	Corse-du-Sud	4 062,48	
10	Gironde	3 806,8	

REQUETE 5 :

Prix moyen du mètre carré d'une maison en Île-de-France

```
SELECT  
ROUND(AVG(valeur_fonciere /  
surface_carrez), 2) AS "prix_m²"  
FROM jointure  
WHERE loc_type = "Maison"  
AND reg_nom = "Ile-de-France";
```

	123 prix_m²	
1	3764,39	

REQUETE 6 :

Liste des 10 appartements les plus chers avec la région et le nombre de mètres carrés

```
SELECT
id_bien,
reg_nom,
surface_carrez,
valeur_fonciere
FROM jointure
WHERE loc_type = "Appartement"
ORDER BY valeur_fonciere DESC
LIMIT 10;
```

	A-Z reg_nom ▼	123 surface_carre ▼	123 valeur_fonciere ▼
1	Ile-de-France	9,1	9 000 000
2	Ile-de-France	64	8 600 000
3	Ile-de-France	20,55	8 577 713
4	Ile-de-France	42,77	7 620 000
5	Ile-de-France	253,3	7 600 000
6	Ile-de-France	139,9	7 535 000
7	Ile-de-France	360,95	7 420 000
8	Ile-de-France	595	7 200 000
9	Ile-de-France	122,56	7 050 000
10	Ile-de-France	79,38	6 600 000

REQUETE 7 :

Taux d'évolution du nombre de ventes entre le premier et le second trimestre de 2020

```
WITH trimestres AS (  
  SELECT  
    COUNT(CASE WHEN date_mutation BETWEEN  
      "2020/01/01" AND "2020/03/31"  
    THEN 1 END) AS trim_1,  
    COUNT(CASE WHEN date_mutation BETWEEN  
      "2020/04/01" AND "2020/06/30"  
    THEN 1 END) AS trim_2  
  FROM jointure)  
SELECT trim_1, trim_2,  
  ROUND(((CAST(trim_2 AS FLOAT) - trim_1)  
    / trim_1) * 100, 2) || " %" AS  
  taux_evolution  
FROM trimestres;
```

	123 trim_1	123 trim_2	A-Z taux_evolution
1	16 776	17 393	3.68 %

REQUETE 8 :

Le classement des régions par rapport au prix au mètre carré des appartements de plus de 4 pièces

```
SELECT
reg_nom,
ROUND(AVG(valeur_fonciere /
surface_carrez), 2) AS "prix_m²"
FROM jointure
WHERE loc_type = "Appartement"
AND total_piece > 4
GROUP BY reg_nom
ORDER BY "prix_m²" DESC;
```

	A-Z reg_nom	123 prix_m²
1	Ile-de-France	8 806,2
2	La Réunion	3 659
3	Provence-Alpes-Côte d'Azur	3 616,21
4	Corse	3 117,2
5	Auvergne-Rhône-Alpes	2 903,36
6	Nouvelle-Aquitaine	2 476,01
7	Bretagne	2 426,7
8	Pays de la Loire	2 328,7
9	Hauts-de-France	2 199,43
10	Occitanie	2 106,77
11	Normandie	2 025,88
12	Grand Est	1 560,43
13	Centre-Val de Loire	1 459,52
14	Bourgogne-Franche-Comté	1 260,42
15	Martinique	574

REQUETE 9 :

Liste des communes ayant eu au moins 50 ventes au 1er trimestre

```
SELECT
com_nom,
COUNT(*) AS nombre_ventes
FROM jointure
WHERE date_mutation BETWEEN
"2020/01/01" AND "2020/03/31"
GROUP BY com_nom, com_code
HAVING nombre_ventes >= 50;
```

	AZ com_nom	123 nombre_ventes
1	Ajaccio	54
2	Angers	64
3	Antibes	77
4	Asnières-sur-Seine	81
5	Bordeaux	157
6	Boulogne-Billancourt	99
7	Courbevoie	80
8	Grenoble	106
9	Issy-les-Moulineaux	50
10	La Ciotat	62
11	Levallois-Perret	59
12	Lille	67
13	Marseille 1er Arrondissement	71
14	Marseille 4e Arrondissement	72
15	Marseille 8e Arrondissement	81
16	Marseille 9e Arrondissement	66
17	Montreuil	65
18	Nantes	119
19	Nice	173
20	Nîmes	63
21	Paris 10e Arrondissement	109
22	Paris 11e Arrondissement	169
23	Paris 12e Arrondissement	110
24	Paris 13e Arrondissement	94

	AZ com_nom	123 nombre_ventes
24	Paris 13e Arrondissement	94
25	Paris 14e Arrondissement	146
26	Paris 15e Arrondissement	215
27	Paris 16e Arrondissement	165
28	Paris 17e Arrondissement	228
29	Paris 18e Arrondissement	209
30	Paris 19e Arrondissement	116
31	Paris 20e Arrondissement	127
32	Paris 2e Arrondissement	61
33	Paris 3e Arrondissement	79
34	Paris 4e Arrondissement	60
35	Paris 5e Arrondissement	79
36	Paris 6e Arrondissement	86
37	Paris 7e Arrondissement	87
38	Paris 8e Arrondissement	62
39	Paris 9e Arrondissement	106
40	Puteaux	53
41	Rennes	61
42	Rueil-Malmaison	68
43	Saint-Maur-des-Fossés	56
44	Sète	62
45	Toulon	59
46	Toulouse	78
47	Versailles	54
48	Vincennes	68

REQUETE 10 :

Différence en pourcentage du prix au mètre carré entre un appartement de 2 pièces et un appartement de 3 pièces

```
WITH diff_appt AS (  
  SELECT  
    ROUND(AVG(CASE WHEN loc_type = "Appartement"  
      AND total_piece = 2 THEN 1.0 *  
      valeur_fonciere / surface_carrez END)) AS t2,  
    ROUND(AVG(CASE WHEN loc_type = "Appartement"  
      AND total_piece = 3 THEN 1.0 *  
      valeur_fonciere / surface_carrez END)) AS t3  
    FROM jointure  
  )  
  SELECT t2, t3,  
    ROUND(((CAST(t3 AS FLOAT) - t2) / t2) * 100, 2)  
    || " %" AS diff_pourc  
  FROM diff_appt;
```

123 t2 ▼	123 t3 ▼	A-Z diff_pourc ▼
4965	4336	-12.67 %

REQUETE 11 :

Les moyennes de valeurs foncières pour le top 3 des communes des départements 6, 13, 33, 59 et 69

```
SELECT "6" AS dep_code, com_nom, moyenne
FROM (
  SELECT com_nom, ROUND(AVG(1.0 *
    valeur_fonciere)) AS moyenne
  FROM jointure
  WHERE dep_code="6"
  GROUP BY com_nom
  ORDER BY moyenne DESC
  LIMIT 3
)
UNION ALL
SELECT "13" AS dep_code, com_nom, moyenne
FROM (
  SELECT com_nom, ROUND(AVG(1.0 *
    valeur_fonciere)) AS moyenne
  FROM jointure
  WHERE dep_code="13"
  GROUP BY com_nom
  ORDER BY moyenne DESC
  LIMIT 3
)
UNION ALL
SELECT "33" AS dep_code, com_nom, moyenne
FROM (
  SELECT com_nom, ROUND(AVG(1.0 *
    valeur_fonciere)) AS moyenne
  FROM jointure
  WHERE dep_code="33"
  GROUP BY com_nom
  ORDER BY moyenne DESC
  LIMIT 3
)
UNION ALL
SELECT "59" AS dep_code, com_nom, moyenne
FROM (
  SELECT com_nom, ROUND(AVG(1.0 *
    valeur_fonciere)) AS moyenne
  FROM jointure
  WHERE dep_code="59"
  GROUP BY com_nom
  ORDER BY moyenne DESC
  LIMIT 3
)
UNION ALL
SELECT "69" AS dep_code, com_nom, moyenne
FROM (
  SELECT com_nom, ROUND(AVG(1.0 *
    valeur_fonciere)) AS moyenne
  FROM jointure
  WHERE dep_code="69"
  GROUP BY com_nom
  ORDER BY moyenne DESC
  LIMIT 3
)
);
```

	AZ dep_code ▼	AZ com_nom ▼	123 moyenne ▼
1	6	Saint-Jean-Cap-Ferrat	968 750
2	6	Eze	655 000
3	6	Mouans-Sartoux	476 898
4	13	Gignac-la-Nerthe	330 000
5	13	Saint-Savournin	314 425
6	13	Cassis	313 417
7	33	Lège-Cap-Ferret	549 501
8	33	Vayres	335 000
9	33	Arcachon	307 436
10	59	Bersée	433 202
11	59	Cysoing	408 550
12	59	Halluin	322 250
13	69	Ville-sur-Jarnioux	485 300
14	69	Lyon 2e Arrondissement	455 217
15	69	Lyon 6e Arrondissement	426 968

REQUETE 12 :

Les 20 communes avec le plus de transactions pour 1000 habitants pour les communes qui dépassent les 10 000 habitants

```
SELECT com_nom, population,  
ROUND((1.0 * COUNT(*) / population *  
1000.0 ),2) AS "transac/1000"  
FROM jointure  
WHERE population > 10000  
GROUP BY com_nom, com_code, population  
ORDER BY "transac/1000" DESC  
LIMIT 20;
```

	AZ com_nom	123 population	123 transac/1000
1	Paris 2e Arrondissement	21 735	5,84
2	Paris 1er Arrondissement	16 055	4,92
3	Paris 3e Arrondissement	34 306	4,69
4	Arcachon	11 898	4,62
5	La Baule-Escoublac	16 797	4,58
6	Paris 4e Arrondissement	29 390	4,08
7	Roquebrune-Cap-Martin	13 041	3,99
8	Paris 8e Arrondissement	36 250	3,83
9	Sanary-sur-Mer	17 160	3,5
10	La Londe-les-Maures	10 776	3,43
11	Paris 9e Arrondissement	60 563	3,43
12	Paris 6e Arrondissement	41 171	3,38
13	Saint-Cyr-sur-Mer	11 725	3,24
14	Chantilly	11 178	3,13
15	Pornichet	11 440	3,06
16	Saint-Mandé	22 576	3,06
17	Paris 10e Arrondissement	86 863	3,04
18	Menton	30 981	2,94
19	Saint-Hilaire-de-Riez	11 501	2,87
20	Vincennes	50 230	2,81



Merci !